Metodologia Experimental

Sistema Multiagente Baseado em LLM para Orquestracao de Modulos Legados na RoboCup@Home — Lucca Santos Cerasomma / 113653 — Centro Universitario da Fundacao Herminio Ometto. Documento gerado em 07/10/2026.

Este documento descreve o protocolo experimental adotado na validacao do modulo de raciocinio multiagente, a composicao do conjunto de ferramentas (*toolkit*) exposto ao Agente de Execucao e o mecanismo pelo qual essas ferramentas sao selecionadas e invocadas. Destina-se a compor a secao de Metodologia do trabalho.

1. Ambiente experimental

Componente	Configuracao
Sistema operacional	Ubuntu 22.04 LTS (kernel 6.8)
Middleware robotico	ROS 2 Humble Hawksbill
Simulador	Ignition Gazebo 6.18.0 (Fortress)
Navegacao	Nav2 — planejador global NavFn, controlador local DWB
Localizacao	AMCL sobre mapa estatico (`likelihood_field`, 500–2000 particulas)
Modelo de linguagem	**Llama 3.1 8B**, servido localmente por Ollama; formato GGUF, quantizacao Q4_K_M; temperatura 0
Orquestracao	LangGraph 1.2.11; langchain-core 1.6.0; langchain-ollama 1.1.0
Linguagem	Python 3.10.12

Cenario. Residencia simulada de 30 x 27 m (Cenario1), dividida em quatro quadrantes por divisorias internas e mobiliada com mesas, armario e obstaculos cilindricos. O robo e de tracao diferencial, raio 0,2 m, equipado com LiDAR 2D operando a 10 Hz. A posicao de origem (*spawn*) e (-3,9; -5,8) no referencial do mundo.

Mapa. Utilizou-se um mapa previamente levantado por SLAM Toolbox (`Cenario1_12_11_10h11min`, 29,1 x 26,1 m, resolucao 0,05 m/celula), carregado estaticamente pelo `map_server`. A origem do referencial `map` coincide com o *spawn* do robo, de modo que as coordenadas do mapa e as do mundo diferem por um deslocamento constante de (-3,9; -5,8), aplicado explicitamente nas conversoes.

Parametros de navegacao. Velocidade linear maxima 0,26 m/s; velocidade angular maxima 1,0 rad/s; tolerancia de chegada 0,50 m em posicao e 3,14 rad em orientacao; raio de inflacao 0,55 m. A tolerancia angular ampla reflete o fato de que a tarefa e deslocar-se ate um *comodo*, nao atingir uma pose exata.

2. Arquitetura do modulo de raciocinio

O modulo foi implementado como um grafo de estados (LangGraph) de malha fechada, com tres agentes

especializados. O Agente Supervisor e o ponto de entrada e o unico despachante: ele nunca executa acoes, apenas decide qual agente recebe o controle em seguida.

```
        SUPERVISOR <---------------+
         |     |      |            |
         v     v      v            |
    PLANEJADOR EXECUTOR  FIM       |
         |     |                   |
         +-----+-------------------+
```

Agente	Responsabilidade	Referencia de projeto
Supervisor	Mantem o estado da missao; despacha uma subtarefa por vez; avalia o feedback do ROS 2; verifica o cumprimento do objetivo; dispara replanejamento	Supervisor do MALMM + Agente de Avaliacao do MARS
Planejador	Decompoe a intencao do usuario em subtarefas sequenciais; gera plano de recuperacao quando recebe contexto de erro	Planner do MALMM
Executor	Traduz uma subtarefa em exatamente uma chamada de ferramenta ROS 2	Agente ReAct do ROSA

A integracao com os modulos legados ocorre por **topicos ROS 2**, nao por chamada de funcao: as metas de navegacao sao publicadas em `/room`, consumido pelo no `navigate_to_goal.py` do modulo de navegacao legado, que por sua vez aciona o Nav2 pela *action* `navigate_to_pose`. O desfecho real retorna pelo topico `/nav_result`. O contrato de integracao e, portanto, a interface de mensageria ja definida pelo trabalho legado.

3. Conjunto de ferramentas (toolkit)

Foram implementadas dez ferramentas, correspondentes a Tabela 3 do pre-projeto. Cada uma e uma funcao Python decorada com `@tool` (LangChain), cuja assinatura e docstring sao convertidas automaticamente no esquema JSON apresentado ao modelo.

Ferramenta	Parametros	Modulo alvo	Verificacao do efeito
`navegar_para`	`destino: str`	Navegacao (Nav2 via `/room`)	pose real do simulador comparada a coordenada da meta
`status_navegacao`	—	Navegacao	leitura; sem efeito
`dar_re`	`distancia_m: float`	Navegacao (`behavior_server`)	deslocamento medido por odometria
`girar`	`graus: float`	Navegacao (`behavior_server`)	estado terminal da *action*

`procurar_pessoa_na_sala`	`nome: str`	Percepcao (simulada)	*ground truth* do cenario
`procurar_objeto_na_sala`	`objeto: str`	Percepcao (simulada)	*ground truth* do cenario
`pegar_objeto`	`objeto: str`	Manipulacao (simulada)	presenca no comodo e limite de carga de 3 kg
`entregar_objeto`	`objeto: str`, `destino: str`	Manipulacao (simulada)	estado de carga do robo
`falar`	`mensagem: str`	Interacao (topico `/tts`)	publicacao confirmada
`consultar_base`	`pergunta: str`	Conhecimento do ambiente	leitura; sem efeito

> **Decisao de projeto.** As ferramentas de percepcao operam **exclusivamente no comodo corrente**, nao realizando busca multi-comodo. Essa restricao e deliberada: a secao 7.2 do pre-projeto critica o `find_person` do sistema legado justamente por concentrar "toda a logica de negocios (buscar comodos, navegar, procurar, repetir)". Mantendo as ferramentas como primitivas atomicas, a varredura da residencia precisa **emergir do plano gerado pelo Planejador** — que e precisamente a capacidade que o experimento se propoe a medir.

4. Mecanismo de selecao e invocacao de ferramentas

A selecao da ferramenta e feita por *tool calling* nativo, e nao por interpretacao de texto livre. O procedimento, por subtarefa, e o seguinte.

4.1 Exposicao do esquema ao modelo

No inicio da execucao, o conjunto de ferramentas e vinculado ao modelo por `ChatOllama(model="llama3.1", temperature=0).bind_tools(TOOLKIT)`. O LangChain converte cada funcao em um esquema JSON derivado da assinatura e da docstring. Para `navegar_para`, o esquema transmitido ao modelo e:

```
{
  "type": "function",
  "function": {
    "name": "navegar_para",
    "description": "Leva o robo ate um comodo do mapa usando o Nav2 e
                    aguarda o resultado real. Use para qualquer deslocamento.
                    'destino' deve ser um dos comodos conhecidos.",
    "parameters": {
      "type": "object",
      "properties": { "destino": { "type": "string" } },
      "required": ["destino"]
    }
  }
}
```

4.2 Emissao da chamada pelo modelo

O Executor submete ao modelo uma mensagem de sistema — contendo os comodos validos e o mapeamento entre tipos de acao e ferramentas — e a subtarefa recebida do Supervisor. O modelo responde com uma estrutura `tool_calls` contendo o nome da ferramenta e os argumentos, e nao com texto livre.

4.3 Camada de guarda deterministica

Antes da execucao, a escolha do modelo e confrontada com indicios lexicais da subtarefa. A guarda corrige dois erros reproduziveis observados no Llama 3.1 8B: a confusao entre acoes semanticamente proximas (selecionar `procurar_objeto_na_sala` para uma subtarefa de "pegar") e a inversao da convencao de sinal no giro ("para a direita" produzindo valor positivo). Toda correcao e contabilizada como metrica. Caso o modelo nao emita chamada alguma, um resolvedor por expressoes regulares assume, de modo que a subtarefa nunca seja convertida em uma acao arbitraria.

4.4 Execucao e retorno

A ferramenta e entao invocada. No caso de `navegar_para`, a cadeia completa e:

```
Executor
  -> navegar_para(destino="kitchen")
     -> ponte ROS 2: publica "kitchen" no topico /room
        -> navigate_to_goal.py: resolve a coordenada no arquivo de metas
           -> Nav2: action navigate_to_pose
        <- publica desfecho em /nav_result  ("success:kitchen" | "error:kitchen:6")
     <- ponte: aguarda o resultado COM O NOME DA META CORRESPONDENTE,
        confronta com a pose real e devolve veredito
  <- "SUCESSO: ..." | "FALHA: ..." | "VAZIO: ..."
```

O prefixo do retorno determina o encaminhamento no grafo: `SUCESSO` avanca o plano; `FALHA` dispara replanejamento; `VAZIO` — busca realizada sem resultado — conclui a subtarefa e prossegue para o proximo passo do mesmo plano, em conformidade com o fluxo descrito na secao 7.2 do pre-projeto, no qual um retorno *not_found* leva o Supervisor ao comodo seguinte e nao a um replanejamento.

5. Protocolo experimental

5.1 Delineamento

Cinco tarefas, duas condicoes (com e sem falha induzida) e cinco repeticoes, totalizando **50 execucoes**, todas com navegacao fisica no simulador.

Identificador	Comando fornecido ao sistema	Pos-condicao verificada
T1_nav	"va ate o kitchen"	robo no comodo *kitchen*
T2_pessoa	"encontre a Ana e avise onde ela esta"	Ana localizada em *kitchen*
T3_objeto	"encontre o remote control"	objeto localizado em *living room*

T4_entrega	"pegue o soda_can no kitchen e leve para o living room"	*soda_can* entregue em *living room*
T5_sequencia	"va ate o bedroom, depois va ate o dining room e avise que terminou"	robo em *dining room* **e** mensagem emitida

5.2 Inducao controlada de falha

Para medir a Taxa de Recuperacao de Falha, a primeira tentativa de navegacao a um comodo designado retorna falha marcada como induzida, simulando passagem obstruida. A falha ocorre uma unica vez por comodo, de modo que o replanejamento tenha como contorna-la. Execucoes em que a falha nao chegou a ser disparada — porque o plano nao passou pelo comodo designado — sao excluidas do calculo da metrica.

5.3 Verificacao do resultado

Este e o ponto metodologico central. **A conclusao da tarefa nao e aquilo que o agente declara.** Cada tarefa possui uma pos-condicao verificada externamente, contra duas fontes independentes do sistema sob teste: a pose verdadeira do robo, lida diretamente do simulador (`/world/empty/pose/info`) e convertida ao referencial do mapa; e o estado do cenario, definido em arquivo de configuracao.

A adocao dessa separacao decorre de observacao empirica: em campanha preliminar, a transformada `map→base_link` — a estimativa que o proprio robo possui de sua posicao — chegou a divergir 18 m da posicao real. Verificar o resultado por essa transformada mediria a confianca do robo em si mesmo, e nao o desfecho da tarefa. A divergencia entre as duas fontes passou a ser registrada como metrica de erro de localizacao.

> **Separacao de papeis.** O **agente** decide utilizando a estimativa propria, como ocorreria em um robo real; o **verificador do experimento** confere contra a verdade do simulador; a diferenca entre ambos constitui o erro de localizacao registrado.

5.4 Procedimento por execucao

1. Retorno do robo a base por **navegacao** (nao por teleporte, que dessincronizaria a odometria da correspondencia de varreduras do LiDAR).
2. Registro do erro de localizacao acumulado, **antes** de qualquer correcao.
3. Reancoragem da pose estimada por `/initialpose`, equivalente a informar a um robo real sua posicao inicial.
4. Reinicializacao do estado de manipulacao e rearme da falha induzida.
5. Execucao da missao pelo grafo de agentes, com teto de seis replanejamentos.
6. Verificacao da pos-condicao contra o *ground truth*.
7. Registro da pose final real, do erro de localizacao e da ocorrencia de tombamento do robo (altura superior a 0,55 m).

5.5 Metricas coletadas

Metrica	Definicao	Instrumentacao
TCR	Proporcao de tarefas multi-etapa concluidas sem intervencao humana	pos-condicao verificada externamente
FRR	Proporcao de falhas induzidas das quais o Supervisor se recuperou, concluindo a missao	pos-condicao, restrita as execucoes com falha efetivamente disparada
PL	Tempo que o Planejador leva para decompor uma tarefa	cronometragem do no do grafo; decomposicao inicial e replanejamentos registrados separadamente
AL	Intervalo entre o Supervisor receber feedback de sucesso e despachar a subtarefa seguinte ao Executor	marcacao de instante na confirmacao e fechamento no despacho

Registraram-se ainda, por execucao: numero de replanejamentos, chamadas de ferramenta, correcoes da guarda deterministica, erro de localizacao antes e depois da reancoragem, pose final real e ocorrencia de tombamento. Foi tambem instrumentada a **taxa de discordancia** — proporcao de execucoes em que o agente declarou sucesso sem que a pos-condicao se sustentasse — adotada como medida direta da fidedignidade do relato do sistema.

6. Ameacas a validade

1. **Ausencia de reinicializacao completa do simulador entre execucoes.** O servico de reinicializacao de mundo do Ignition Fortress nao restaura poses de modelo, e o teleporte dessincroniza a odometria. Optou-se pela reancoragem da pose estimada no inicio de cada execucao, registrando-se a deriva acumulada previa.
2. **Percepcao e manipulacao simuladas.** O ambiente nao dispoe de modulo de visao computacional nem de manipulador; essas ferramentas respondem contra o estado declarado do cenario, como no sistema legado. **A navegacao, contudo, e fisica.** As metricas medem, portanto, a capacidade de raciocinio somada a navegacao real, e nao acuracia de percepcao.
3. **Dimensao da amostra.** Cinco repeticoes por condicao permitem identificar tendencia, mas nao produzem intervalo de confianca estreito.
4. **Cenario e modelo unicos.** Nao se realizou comparacao entre diferentes residencias simuladas nem entre modelos de linguagem.
5. **Ausencia de linha de base quantitativa do sistema legado.** A comparacao com o pipeline reativo permanece argumentativa — ele nao decompoe tarefas nem replaneja, por construcao — e nao medida lado a lado nas mesmas tarefas.